

Merge Sort

COP 3502C – Computer Science I

Spring 2026

Yancy Vance Paredes, PhD

Recall

- We previously talked about three elementary sorting algorithms
- These sorting algorithms were simple to implement
- However, we must note their worst-case running time

Discussion

- We are going to figure out if we can do better
- But before that, let's look at this scenario

Scenario /1

- Let's say you have **two sorted arrays** of equal length (i.e., size)
- Assume all the elements are **distinct**
- The goal is to **combine** them into one big array
- You must preserve the **sorted property**

Scenario /2

What is your proposed strategy?

10	20	30
----	----	----

15	25	35
----	----	----

10	20	30
----	----	----

15	25	35
----	----	----

--	--	--	--	--	--

10	20	30
----	----	----

15	25	35
----	----	----

i

j

--	--	--	--	--	--

10	20	30
----	----	----

15	25	35
----	----	----

i

j

--	--	--	--	--	--

p?

10	20	30	15	25	35
----	----	----	----	----	----

i

j

10					
----	--	--	--	--	--

p_i

10	20	30	15	25	35
----	----	----	----	----	----

i

j

10					
----	--	--	--	--	--

p_i

10	20	30	15	25	35
----	----	----	----	----	----

i

j

10					
----	--	--	--	--	--

P?

10	20	30
----	----	----

15	25	35
----	----	----

i

j

10	15				
----	----	--	--	--	--

p_j

10	20	30
----	----	----

i

15	25	35
----	----	----

j

10	15				
----	----	--	--	--	--

p_j

10	20	30
----	----	----

i

15	25	35
----	----	----

j

10	15				
----	----	--	--	--	--

p?

10	20	30
----	----	----

i

15	25	35
----	----	----

j

10	15	20			
----	----	----	--	--	--

p_i

10	20	30
----	----	----

15	25	35
----	----	----

i

j

10	15	20			
----	----	----	--	--	--

p_i

10	20	30
----	----	----

15	25	35
----	----	----

i

j

10	15	20			
----	----	----	--	--	--

p?

10	20	30
----	----	----

15	25	35
----	----	----

i

j

10	15	20	25		
----	----	----	----	--	--

p_j

10	20	30
----	----	----

15	25	35
----	----	----

i

j

10	15	20	25		
----	----	----	----	--	--

p_j

10	20	30
----	----	----

15	25	35
----	----	----

i

j

10	15	20	25		
----	----	----	----	--	--

p?

10	20	30	15	25	35
----	----	----	----	----	----

i

j

10	15	20	25	30	
----	----	----	----	----	--

P_i

10	20	30	15	25	35
----	----	----	----	----	----

j

10	15	20	25	30	
----	----	----	----	----	--

p_i

10	20	30
----	----	----

15	25	35
----	----	----

j

10	15	20	25	30	
----	----	----	----	----	--

p?

10	20	30
----	----	----

15	25	35
----	----	----

j

10	15	20	25	30	35
----	----	----	----	----	----

p_j

10	20	30
----	----	----

15	25	35
----	----	----

10	15	20	25	30	35
----	----	----	----	----	----

p_j

10	20	30
----	----	----

15	25	35
----	----	----

10	15	20	25	30	35
----	----	----	----	----	----

Discussion

- What happens if one list is longer than the other?
- Let's assume that **arr1** and **arr2** have lengths of **s1** and **s2**, respectively
- What is the running time of this operation?

Your Turn!

Write a code that accepts two sorted arrays as input along with their corresponding sizes. It returns a single array in which all the elements from the two arrays are combined and are sorted.

```
int *merge(int *arr1, int *arr2, int s1, int s2) {  
  
  
  
  
  
  
  
  
}
```

Discussion

- We recall the Sorting Problem
- Given an array with **N** elements, we want to sort it (e.g., ascending)
- One perspective is to recognize that it is a **big task**
- Why don't we make it **smaller** so that it becomes manageable?

Divide and Conquer

- A strategy in which we **divide** a big task into smaller subproblems
- Continue doing so until it becomes manageable and *small enough*
- We then **conquer** and solve this small subproblem
- Finally, we **combine** the results to solve the overall problem

Merge Sort

- Given an array, find the midpoint to **divide** it into two*
- The size is effectively **halved**: the left and the right sub-arrays
- We **recursively solve** each half by applying merge sort
- Afterward, we **combine** the two sorted sub-arrays into one


```
void merge_sort(int *arr, int start, int end) {
    if( start == end )
        return;
```

```
    int mid = (start+end+1) / 2;
```

→ Divide

```
    merge_sort(arr, start, mid-1); // Left
    merge_sort(arr, mid, end);     // Right
```

```
    merge(arr, start, mid, end);
```

```
}
```

↳ combine

$$T(n) = O(1) + T\left(\frac{n}{2}\right) + T\left(\frac{n}{2}\right) + O(n)$$

$$T(n) = \cancel{O(1)} + 2T\left(\frac{n}{2}\right) + O(n)$$

```

void merge(int *arr, int start, int mid, int end) {
    int size = end-start+1;
    int *aux = malloc(sizeof(int) * size);

    int i = start, j = mid;
    int p = 0;

    while( i < mid && j <= end ) {
        if( arr[i] <= arr[j] )
            aux[p] = arr[i++];
        else
            aux[p] = arr[j++];

        p++;
    }

    // Two exclusive events
    while( i < mid )    aux[p++] = arr[i++];
    while( j <= end )  aux[p++] = arr[j++];

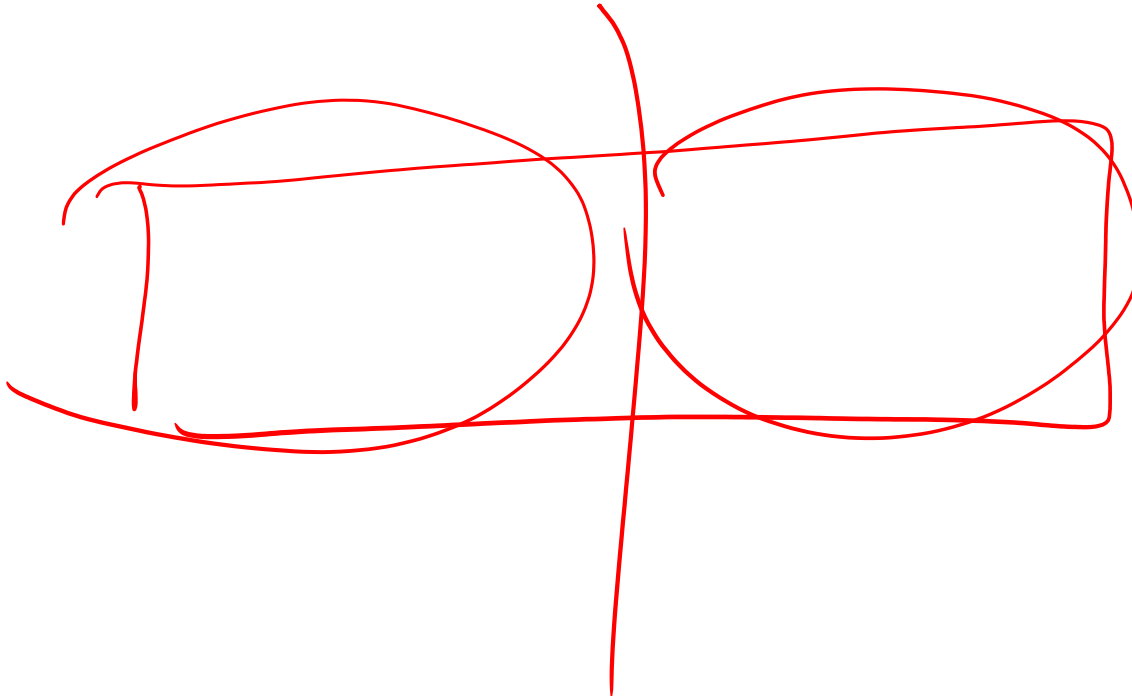
    for(int k = start, pos = 0; k <= end; k++, pos++)
        arr[k] = aux[pos];

    free(aux);
}

```

Visualization /1

- The following is a common way to visualize merge sort
- Notice how the processes are happening in parallel

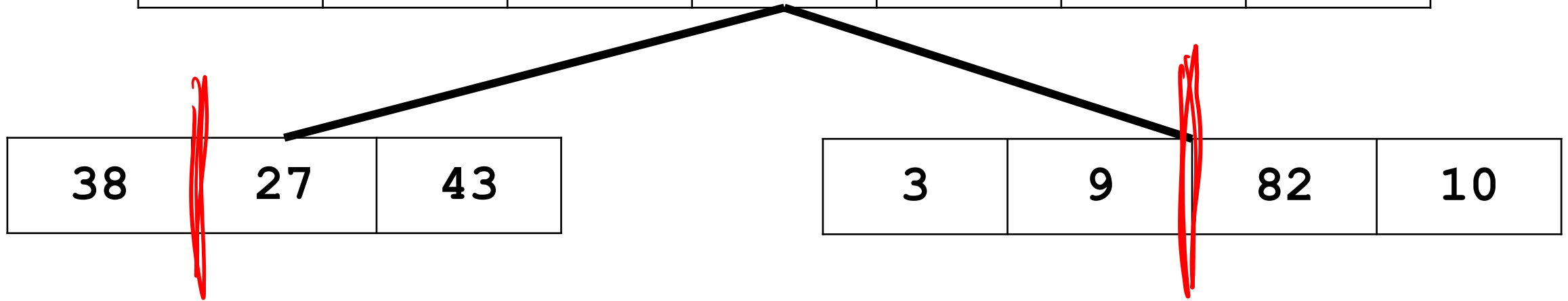


38	27	43	3	9	82	10
----	----	----	---	---	----	----

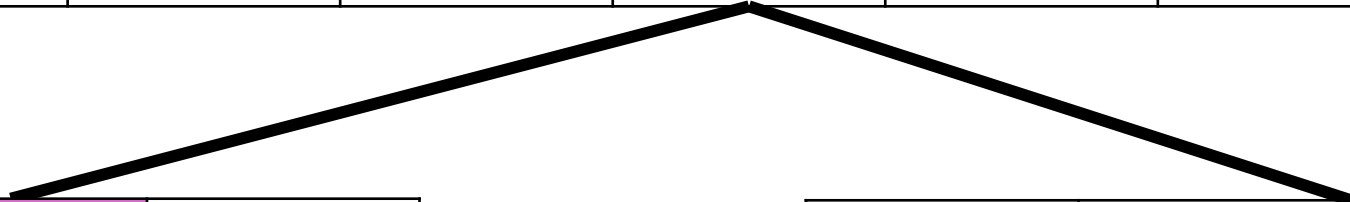


38	27	43	3	9	82	10
----	----	----	---	---	----	----

38	27	43	3	9	82	10
----	----	----	---	---	----	----

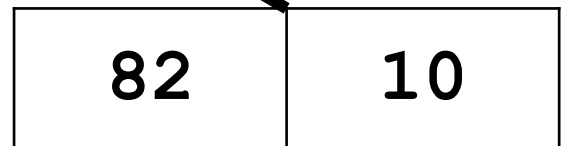
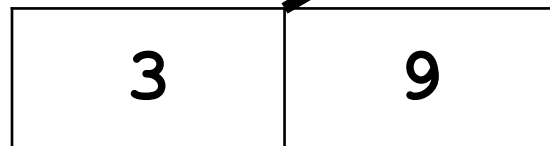
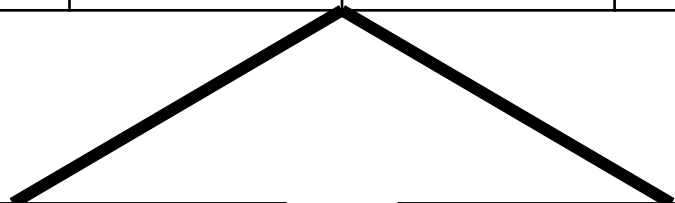
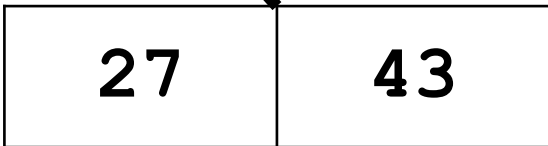
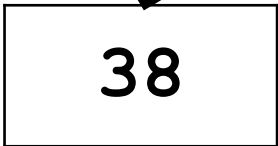
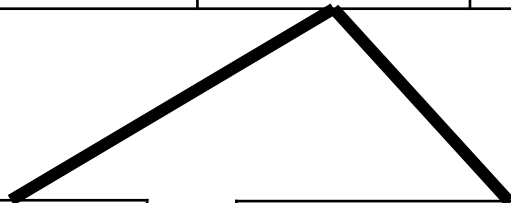
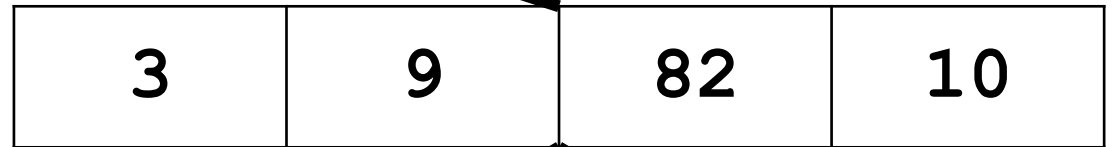
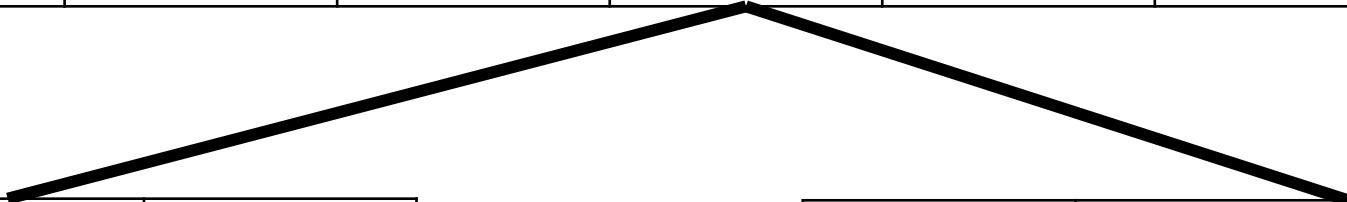
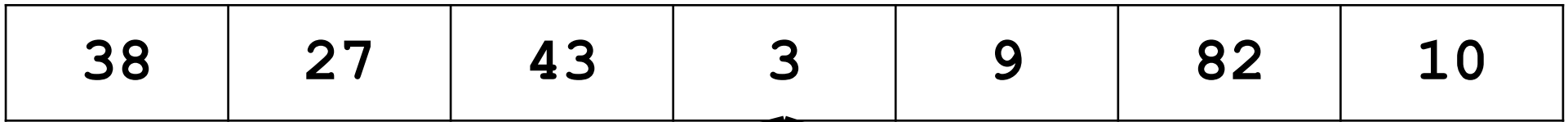


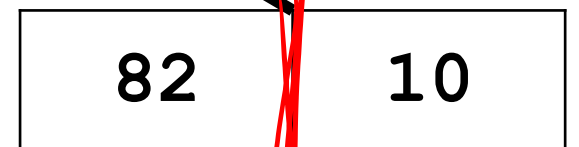
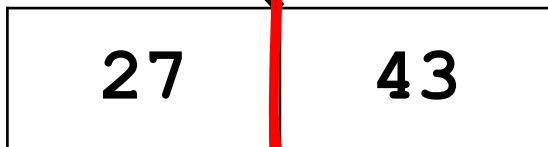
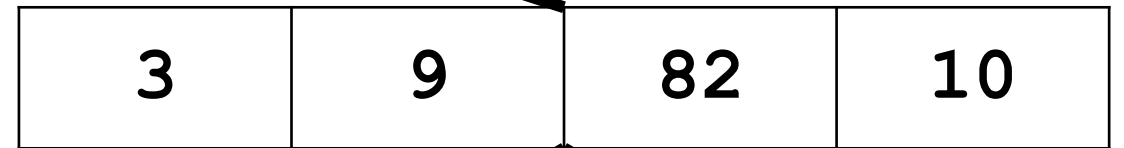
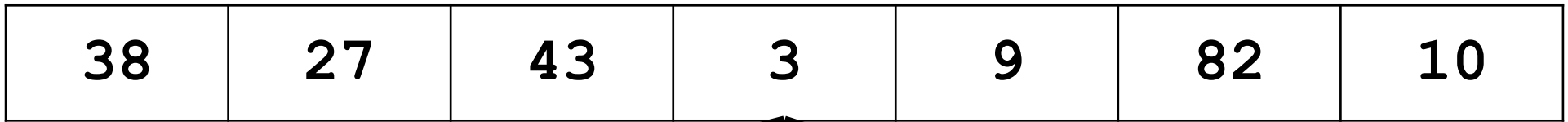
38	27	43	3	9	82	10
----	----	----	---	---	----	----

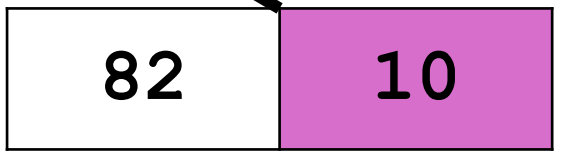
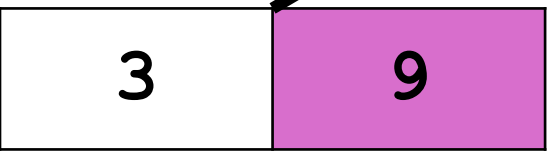
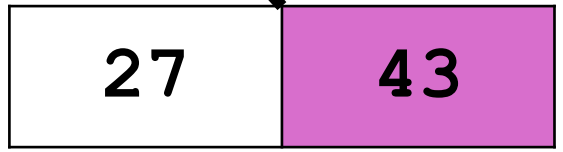
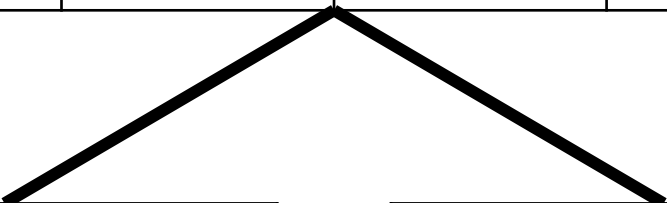
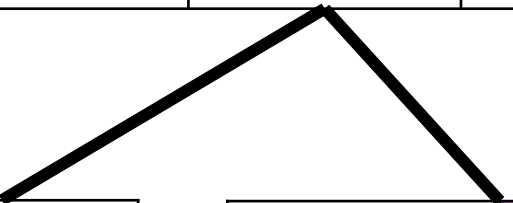
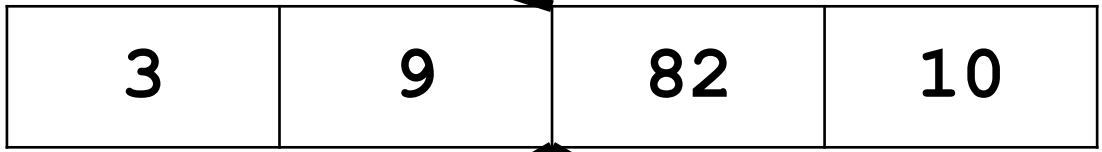
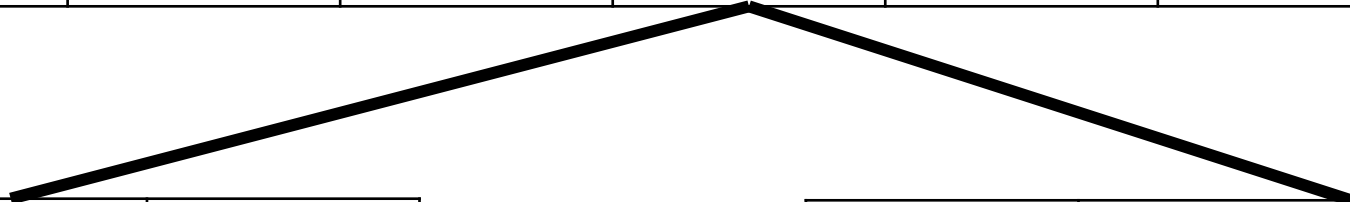
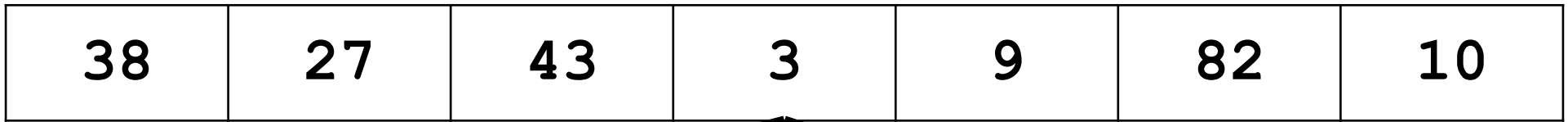


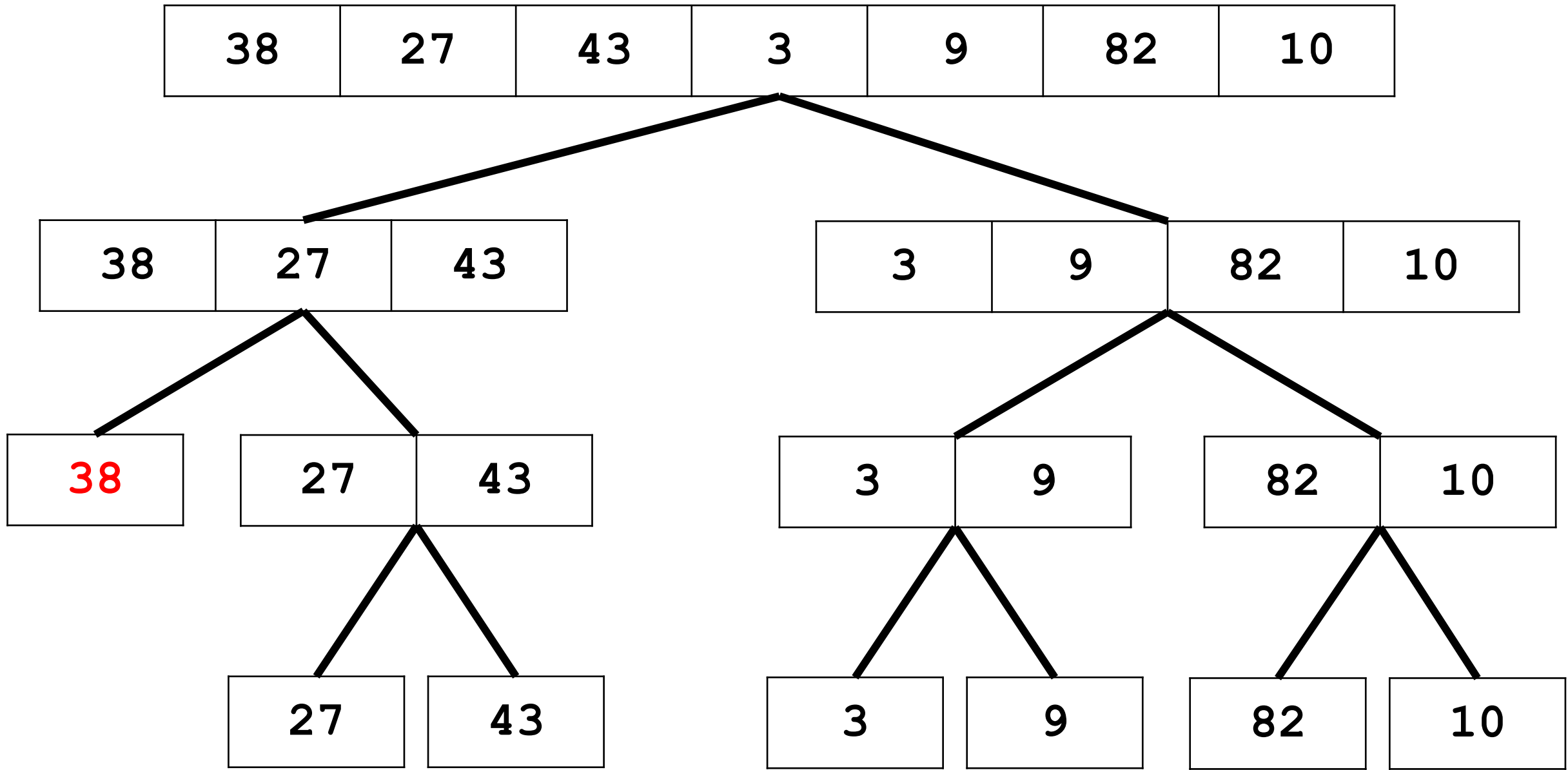
38	27	43
----	----	----

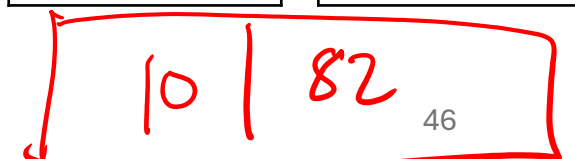
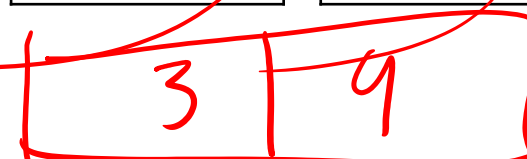
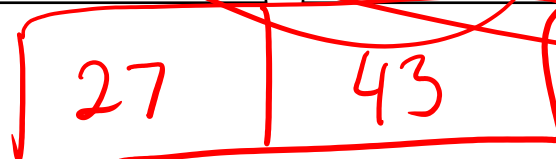
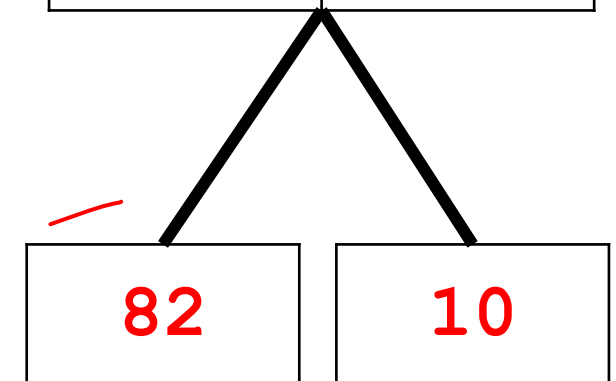
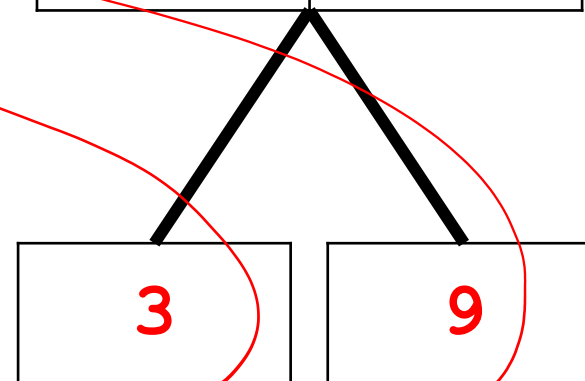
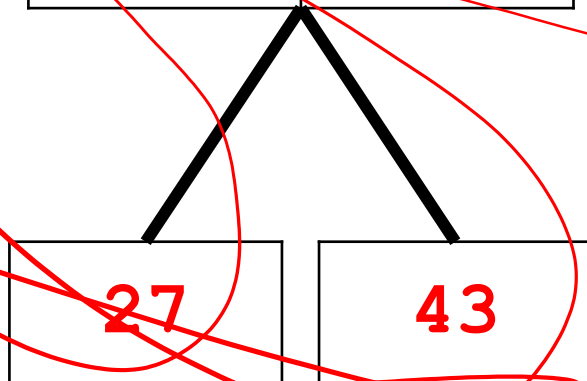
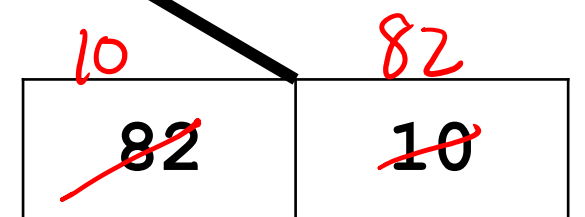
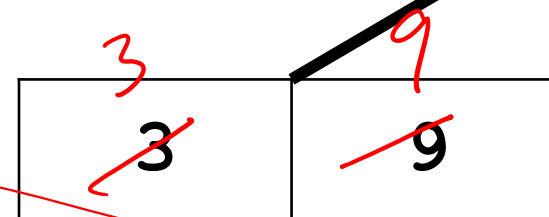
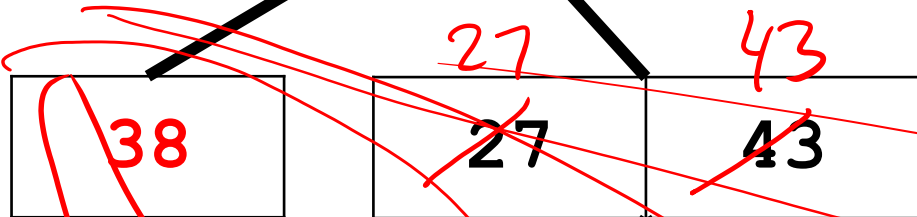
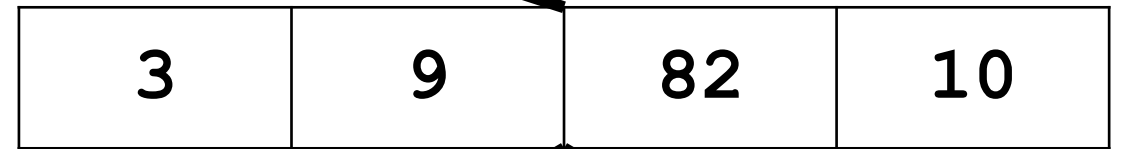
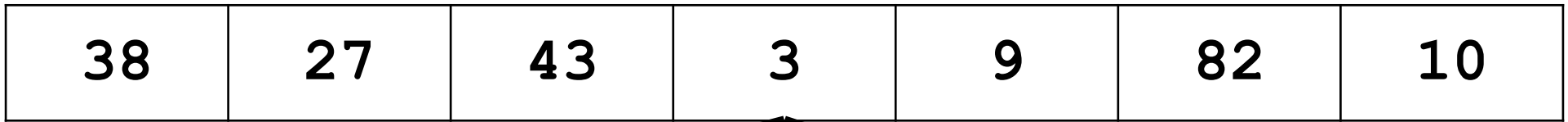
3	9	82	10
---	---	----	----

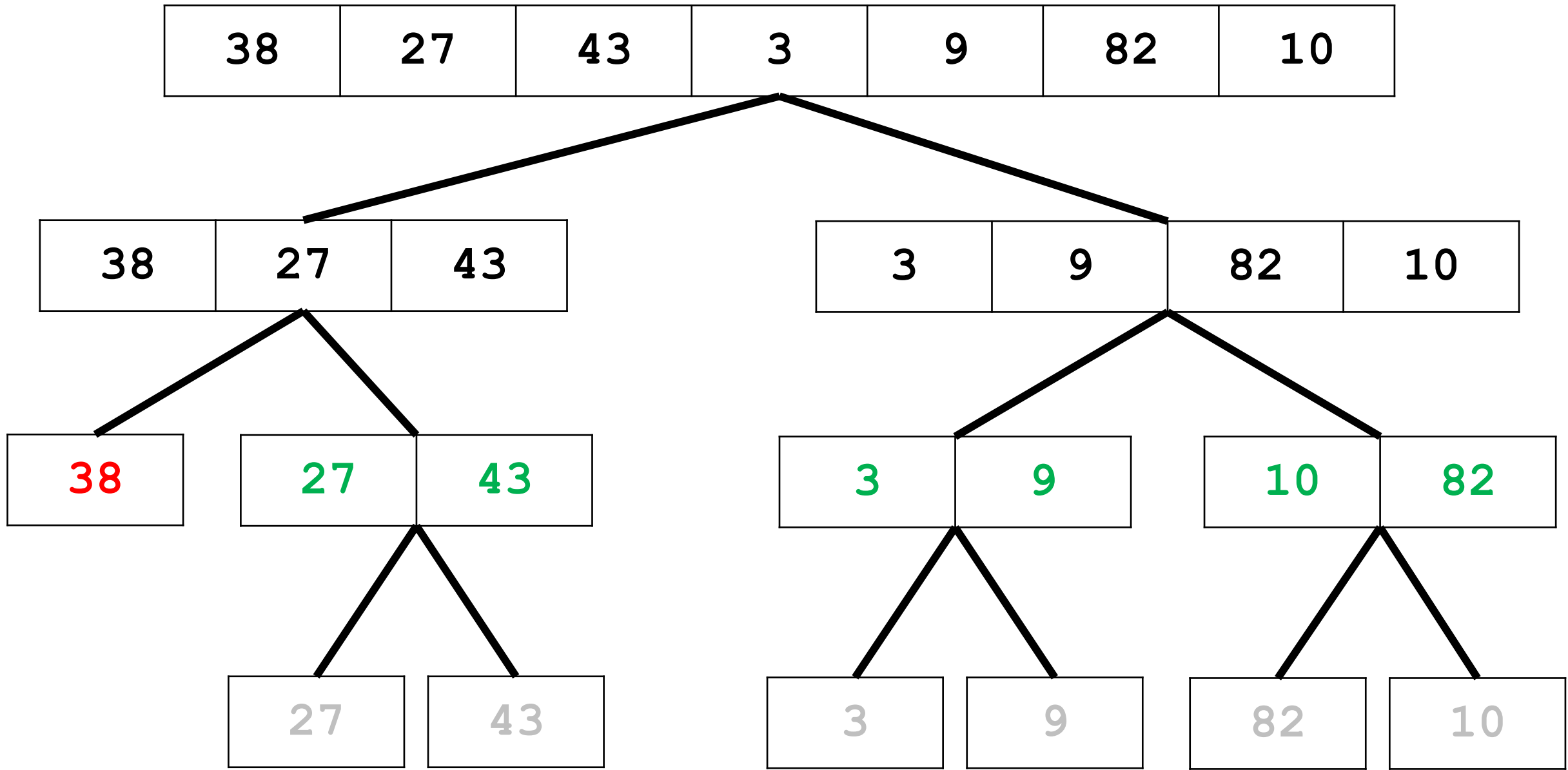




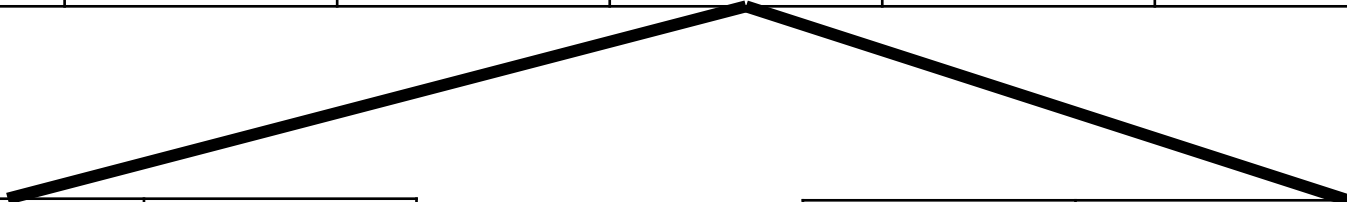








38	27	43	3	9	82	10
----	----	----	---	---	----	----



38	27	43
----	----	----

3	9	82	10
---	---	----	----

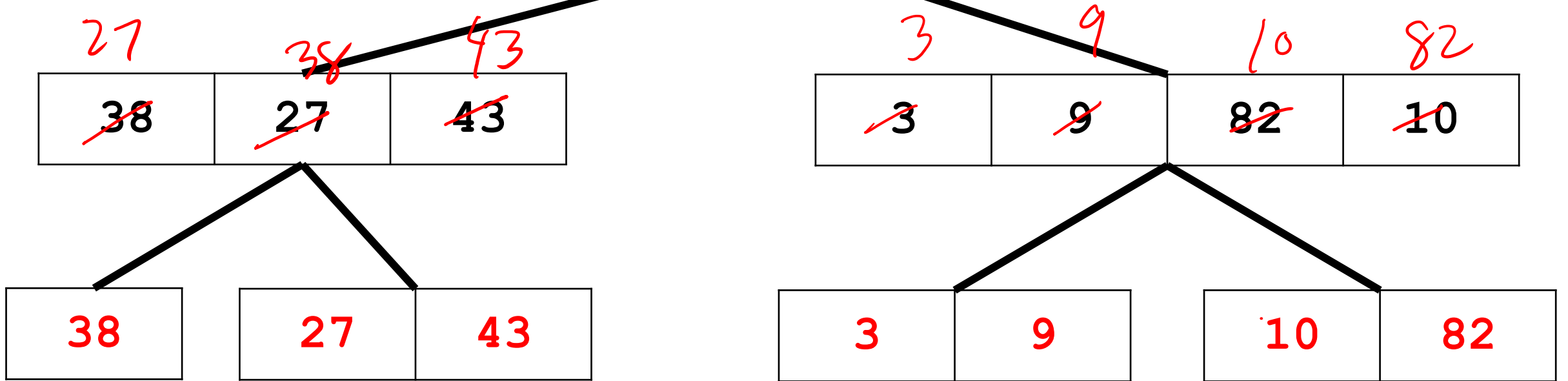
38

27	43
----	----

3	9
---	---

10	82
----	----

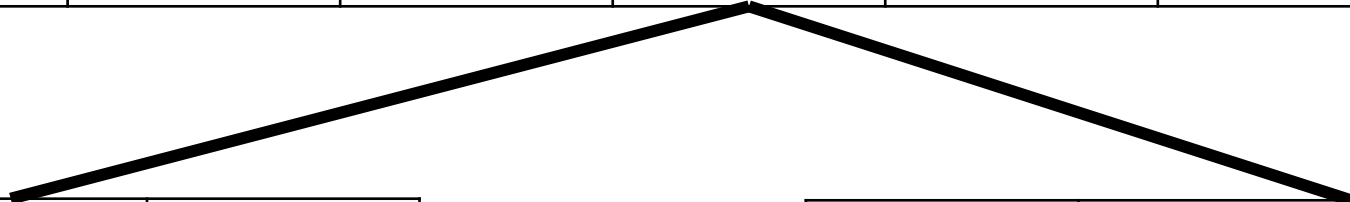
38	27	43	3	9	82	10
----	----	----	---	---	----	----



27	38	43
----	----	----

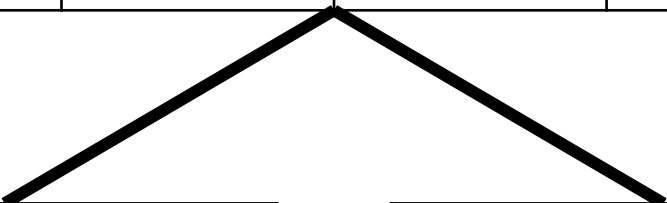
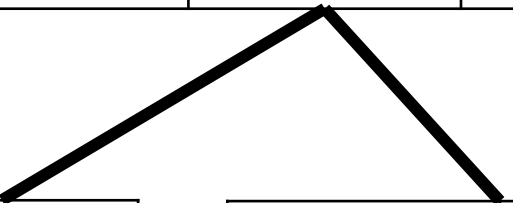
3	9	10	82
---	---	----	----

38	27	43	3	9	82	10
----	----	----	---	---	----	----



27	38	43
----	----	----

3	9	10	82
---	---	----	----



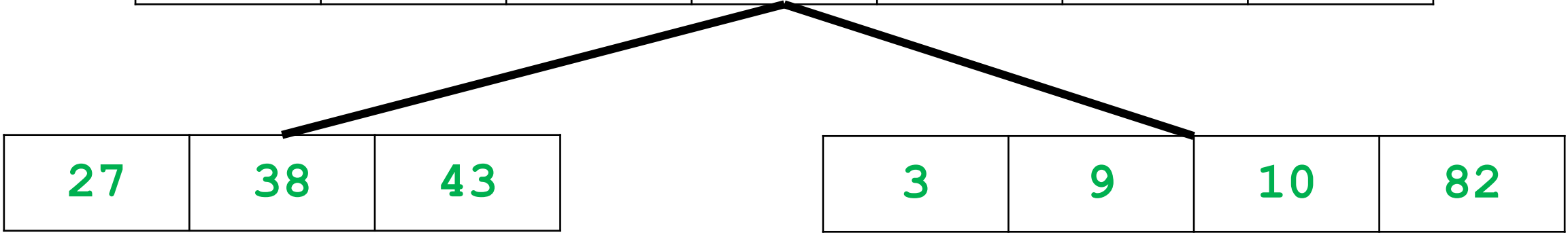
38

27	43
----	----

3	9
---	---

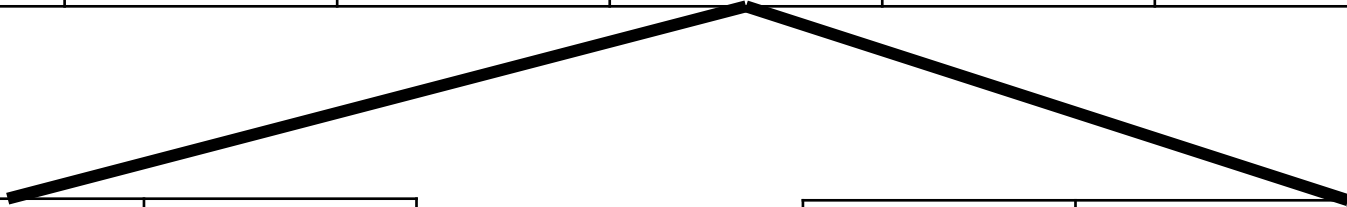
10	82
----	----

³ 38	⁹ 27	¹⁰ 43	²⁷ 3	³⁸ 9	⁴³ 82	⁸² 10
-------------------------------	-------------------------------	--------------------------------	-------------------------------	-------------------------------	--------------------------------	--------------------------------



3	9	10	27	38	43	82
---	---	----	----	----	----	----

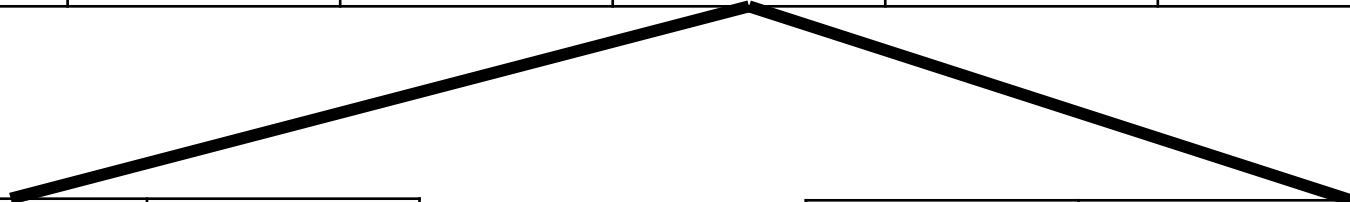
38	27	43	3	9	82	10
----	----	----	---	---	----	----



27	38	43
----	----	----

3	9	10	82
---	---	----	----

3	9	10	27	38	43	82
---	---	----	----	----	----	----



27	38	43
----	----	----

3	9	10	82
---	---	----	----

3	9	10	27	38	43	82
---	---	----	----	----	----	----

3	9	10	27	38	43	82
---	---	----	----	----	----	----

Visualization /2

- The following is an alternative way to visualize merge sort
- It illustrates the non-parallel execution of instructions

38	27	43	3	9	82	10
----	----	----	---	---	----	----

✓
LRC

38	27	43	3	9	82	10
----	----	----	---	---	----	----

38	27	43	3	9	82	10
----	----	----	---	---	----	----

38	27	43
----	----	----

38	27	43	3	9	82	10
----	----	----	---	---	----	----

LRC

38	27	43
----	----	----

✓
LPC

38	27	43	3	9	82	10
----	----	----	---	---	----	----

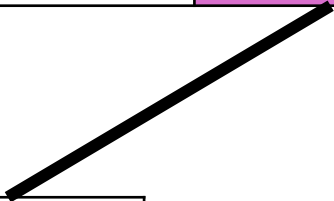
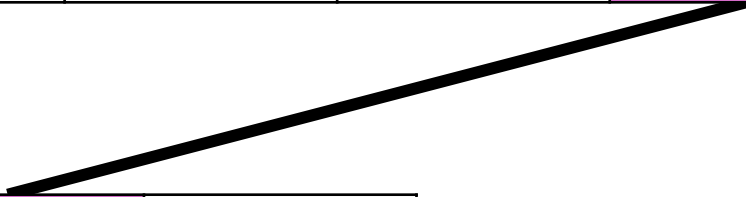
38	27	43
----	----	----

38

38	27	43	3	9	82	10
----	----	----	---	---	----	----

38	27	43
----	----	----

38



38	27	43	3	9	82	10
----	----	----	---	---	----	----

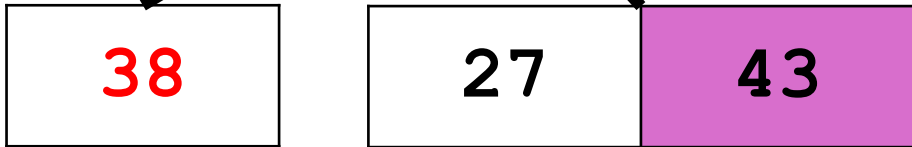
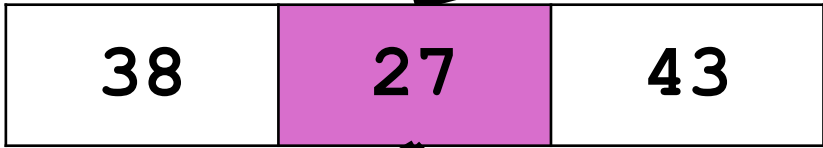
✓✓
LRC

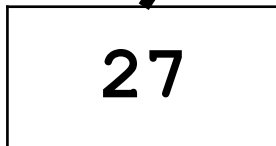
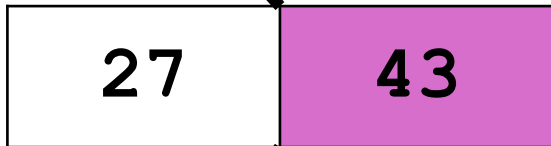
38	27	43
----	----	----

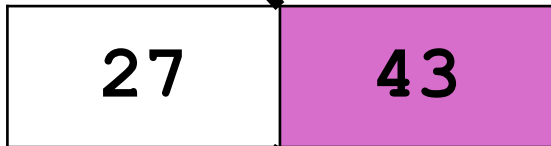
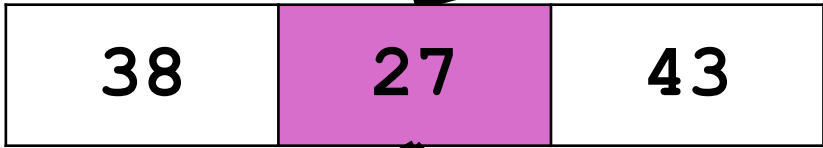
✓
LRC

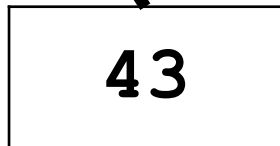
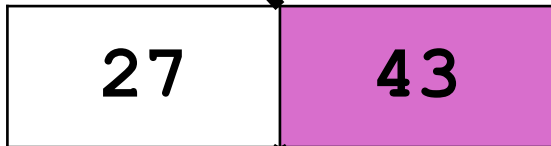
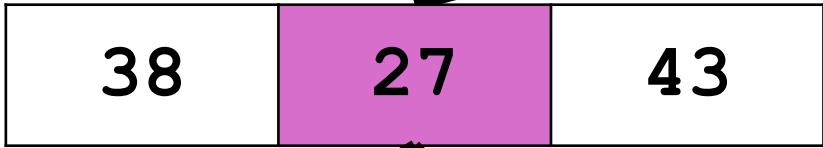
38

27	43
----	----



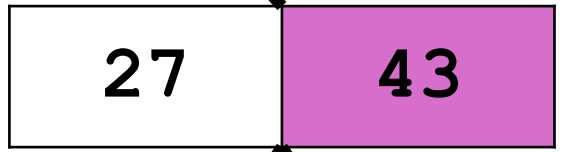


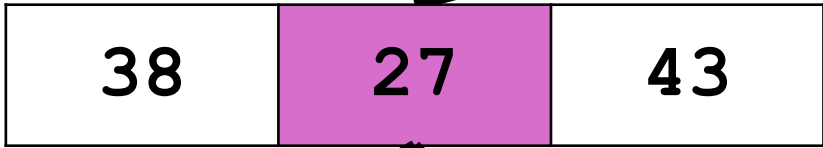






✓✓
LRC





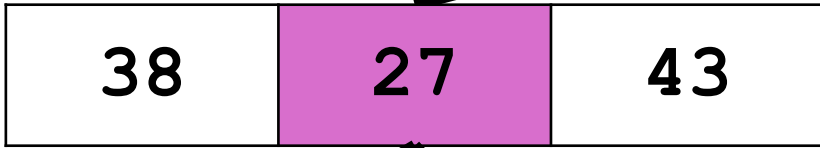
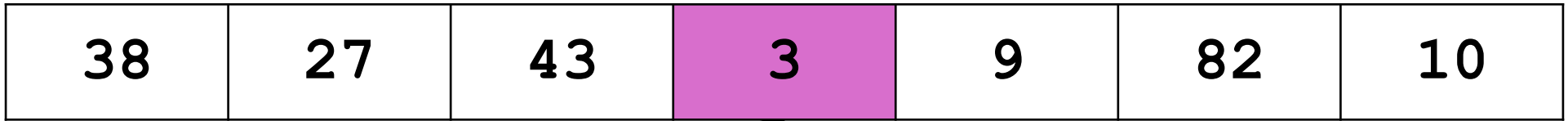
38	27	43	3	9	82	10
----	----	----	---	---	----	----

✓✓
LRC

38	27	43
----	----	----

38

27	43
----	----



38	27	43	3	9	82	10
----	----	----	---	---	----	----

27	38	43
----	----	----

38	27	43
----	----	----

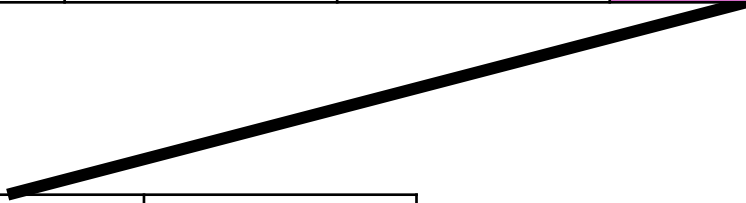
✓
LRC

38	27	43	3	9	82	10
----	----	----	---	---	----	----

27	38	43
----	----	----

38	27	43	3	9	82	10
----	----	----	---	---	----	----

27	38	43
----	----	----



38	27	43	3	9	82	10
----	----	----	---	---	----	----

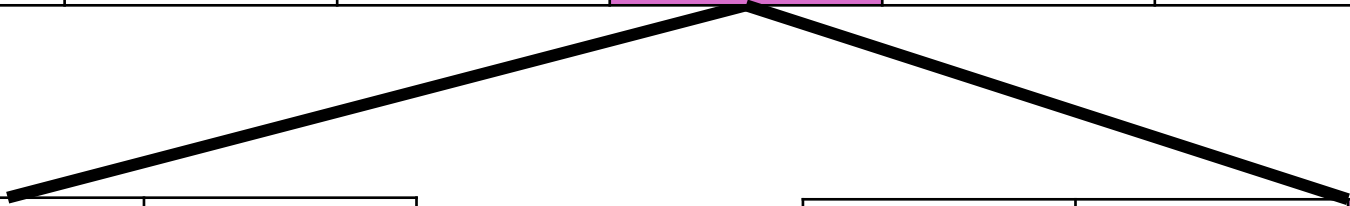


LRC

27	38	43
----	----	----

3	9	82	10
---	---	----	----

38	27	43	3	9	82	10
----	----	----	---	---	----	----



27	38	43
----	----	----

3	9	82	10
---	---	----	----

38	27	43	3	9	82	10
----	----	----	---	---	----	----

27	38	43
----	----	----

3	9	82	10
---	---	----	----

3	9
---	---

38	27	43	3	9	82	10
----	----	----	---	---	----	----

27	38	43
----	----	----

3	9	82	10
---	---	----	----

3	9
---	---

38	27	43	3	9	82	10
----	----	----	---	---	----	----

27	38	43
----	----	----

3	9	82	10
---	---	----	----

3	9
---	---

3

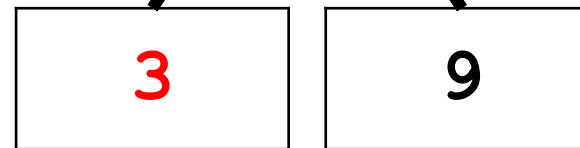
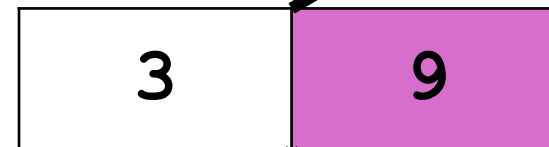
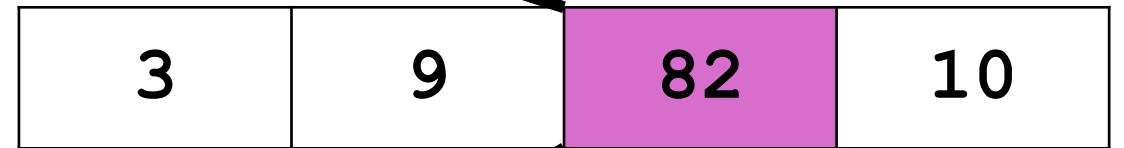
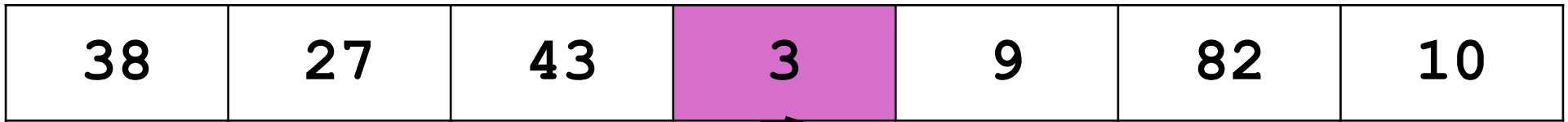
38	27	43	3	9	82	10
----	----	----	---	---	----	----

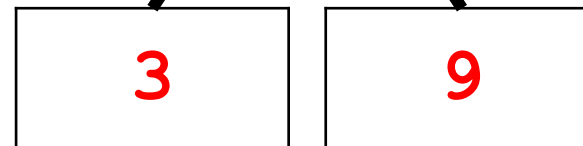
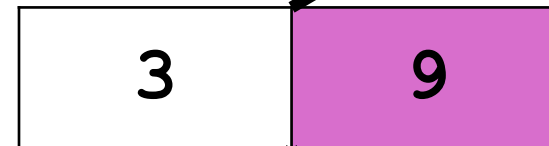
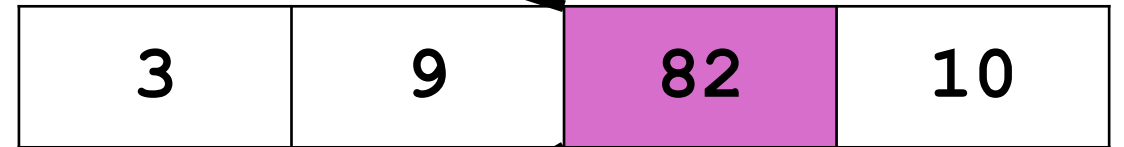
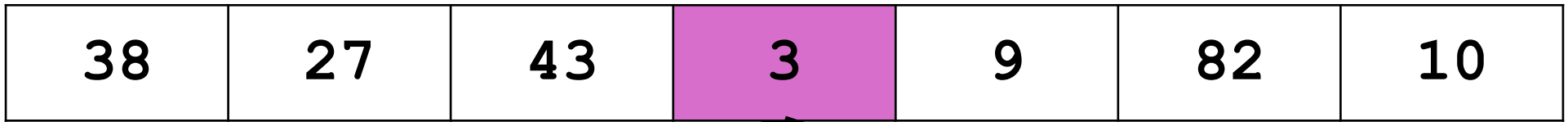
27	38	43
----	----	----

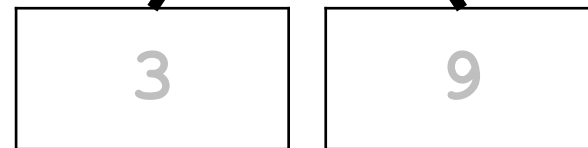
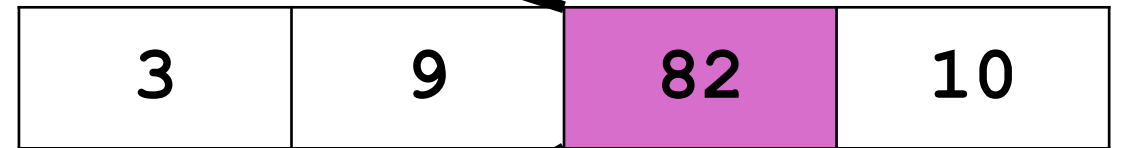
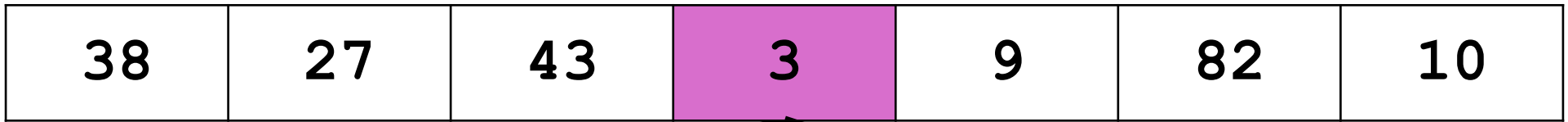
3	9	82	10
---	---	----	----

3	9
---	---

3







38	27	43	3	9	82	10
----	----	----	---	---	----	----

✓
LRC

27	38	43
----	----	----

3	9	82	10
---	---	----	----

3	9
---	---

38	27	43	3	9	82	10
----	----	----	---	---	----	----

27	38	43
----	----	----

3	9	82	10
---	---	----	----

3	9
---	---

38	27	43	3	9	82	10
----	----	----	---	---	----	----

27	38	43
----	----	----

3	9	82	10
---	---	----	----

3	9
---	---

82	10
----	----

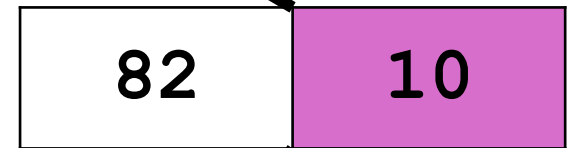
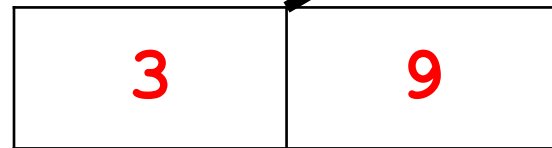
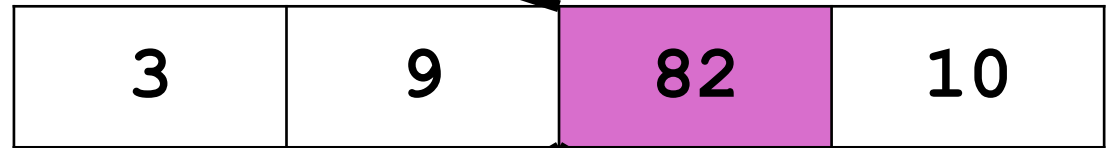
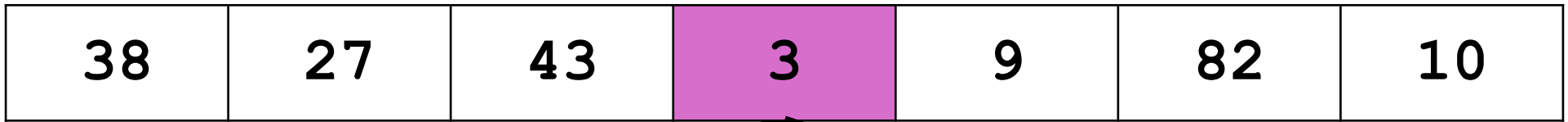
38	27	43	3	9	82	10
----	----	----	---	---	----	----

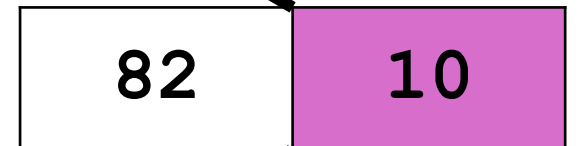
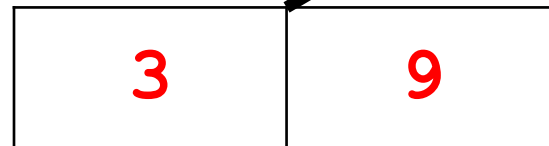
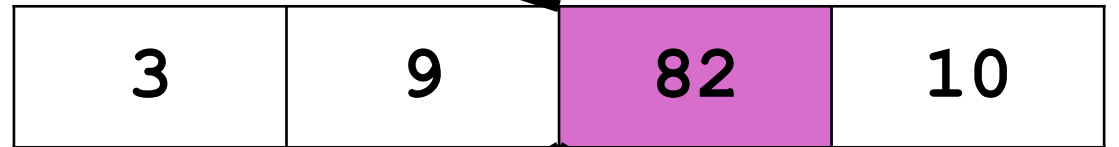
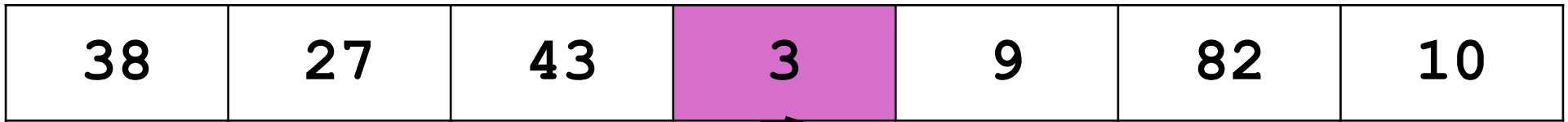
27	38	43
----	----	----

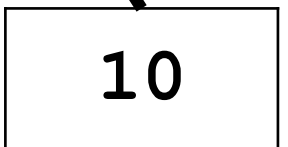
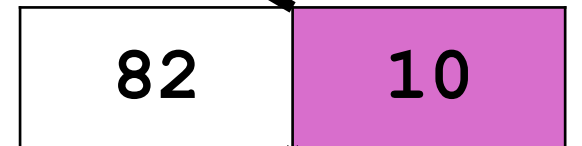
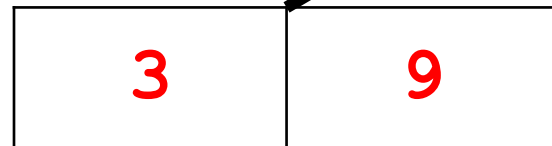
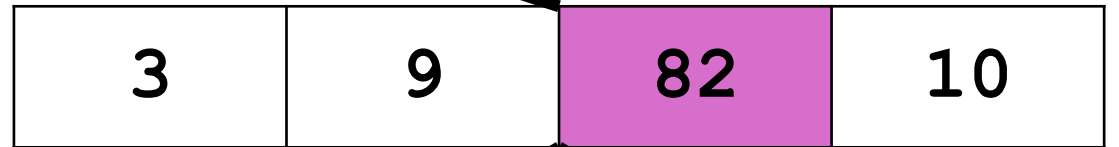
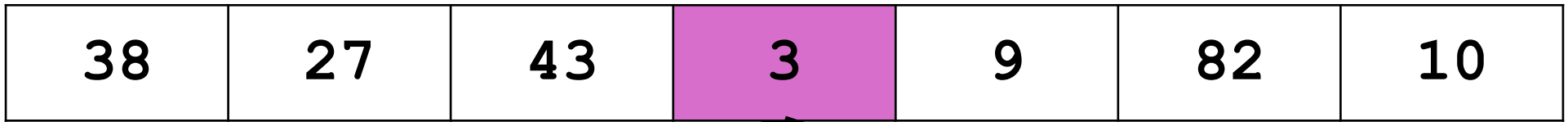
3	9	82	10
---	---	----	----

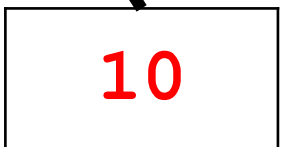
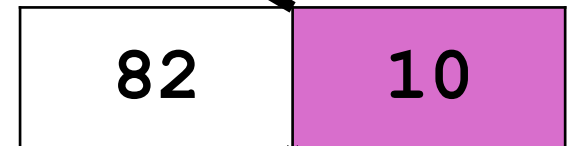
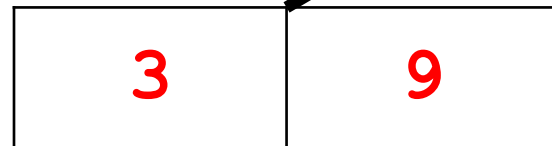
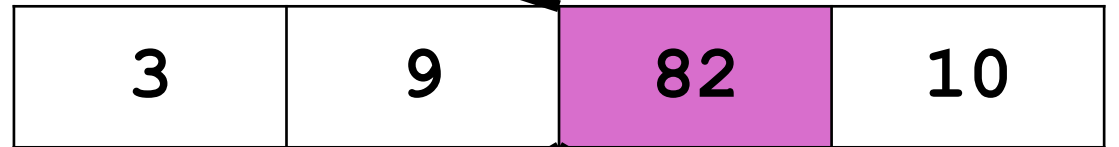
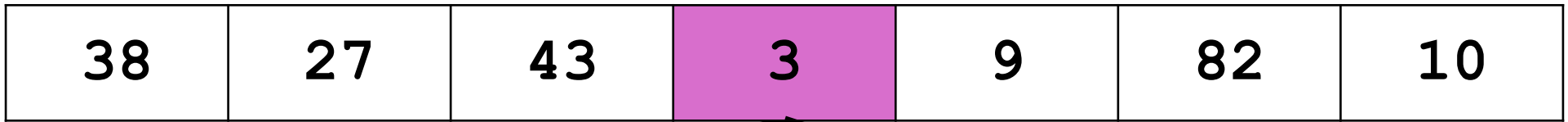
3	9
---	---

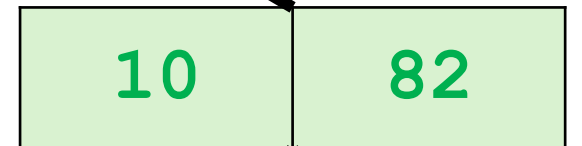
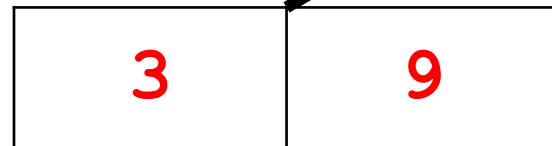
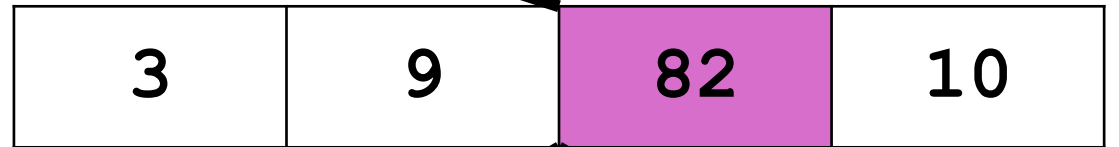
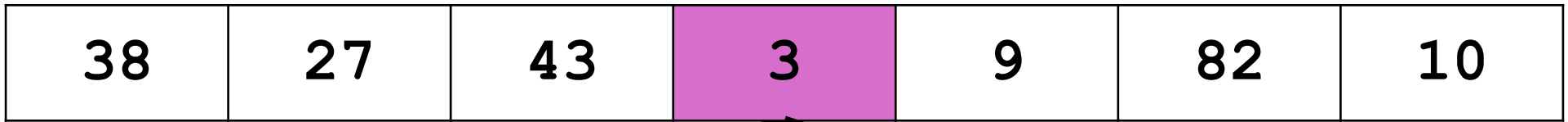
82	10
----	----











38	27	43	3	9	82	10
----	----	----	---	---	----	----

✓✓✓
LRG

27	38	43
----	----	----

3	9	82	10
---	---	----	----

3	9
---	---

10	82
----	----

38	27	43	3	9	82	10
----	----	----	---	---	----	----

27	38	43
----	----	----

3	9	82	10
---	---	----	----

3	9
---	---

10	82
----	----

38	27	43	3	9	82	10
----	----	----	---	---	----	----

27	38	43
----	----	----

3	9	10	82
---	---	----	----

3	9
---	---

10	82
----	----

✓✓✓
LRC

38	27	43	3	9	82	10
----	----	----	---	---	----	----

27	38	43
----	----	----

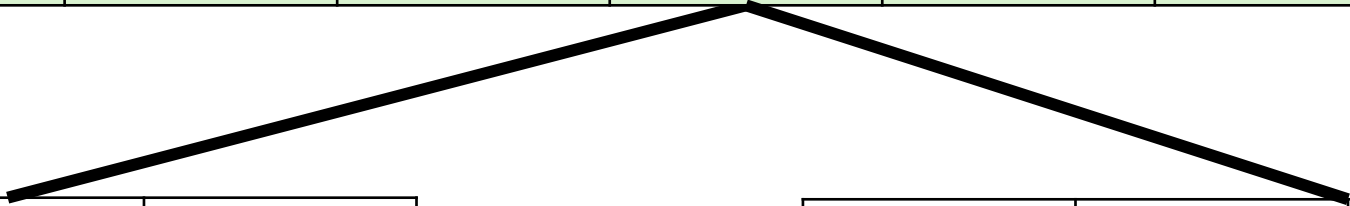
3	9	10	82
---	---	----	----

38	27	43	3	9	82	10
----	----	----	---	---	----	----

27	38	43
----	----	----

3	9	10	82
---	---	----	----

3	9	10	27	38	43	82
---	---	----	----	----	----	----



27	38	43
----	----	----

3	9	10	82
---	---	----	----

3	9	10	27	38	43	82
---	---	----	----	----	----	----

3	9	10	27	38	43	82
---	---	----	----	----	----	----

```

Now: mergesort(0, 6)
L: mergesort(0, 2)
    Now: mergesort(0, 2)
    L: mergesort(0, 0)
        Now: mergesort(0, 0)
            Base: (0)
    R: mergesort(1, 2)
        Now: mergesort(1, 2)
        L: mergesort(1, 1)
            Now: mergesort(1, 1)
                Base: (1)
        R: mergesort(2, 2)
            Now: mergesort(2, 2)
                Base: (2)
    C: merge(1, 2)
C: merge(0, 2)

```

```

R: mergesort(3, 6)
    Now: mergesort(3, 6)
    L: mergesort(3, 4)
        Now: mergesort(3, 4)
        L: mergesort(3, 3)
            Now: mergesort(3, 3)
                Base: (3)
        R: mergesort(4, 4)
            Now: mergesort(4, 4)
                Base: (4)
        C: merge(3, 4)
    R: mergesort(5, 6)
        Now: mergesort(5, 6)
        L: mergesort(5, 5)
            Now: mergesort(5, 5)
                Base: (5)
        R: mergesort(6, 6)
            Now: mergesort(6, 6)
                Base: (6)
        C: merge(5, 6)
    C: merge(3, 6)
C: merge(0, 6)

```

Notes

- The combine step is similar to our earlier scenario
- Can we compute for the number of times **merge ()** was called?

Notes

What is the running time of merge sort?

Discussion

- Notice that the solution involves **recursion**?
- To analyze the running time, we model it using a **recurrence relation**

Space Complexity of Merge Sort

- If implementation uses a temporary array during merging, the required additional space is proportional to **n**
- Therefore, space complexity is **$O(n)$**

Challenge /1

Perform a merge sort on the following list of elements. Illustrate what the array looks like after each merge operation. Further, when the subarray is odd in size, the left side will be shorter than the right side. Hint: There are 10 merge operations.

13, 18, 19, 11, 2, 7, 4, 15, 6, 12, 1

Challenge /2

Perform a merge sort on the following list of elements. Determine how many times the **merge ()** function will be called. Show the content of the array right before the last **merge ()** operation.

17, 12, 11, 14, 15, 16, 10, 13

Final Notes /1

- Initially, it might appear that in the visualization, whenever we divide the array, we are “creating” two sub-arrays (left and right).
- However, we can avoid creating new arrays if we are only dividing the array. We can achieve this by utilizing the indices.
- Using the same array, we simply remember the start and end indices of the left and the right sub-arrays.

Final Notes /2

- This is what was done in our implementation of the merge sort.
- However, note that our implementation of the combine step uses an extra array where we temporarily stored the combined lists. Afterward, we transfer all the elements back to the main array.

Check Your Understanding

Sorting Algorithm	Is Stable?	Worst-Case	Average Case	Best-Case
Selection Sort			$O(n^2)$	
Insertion Sort			$O(n^2)$	
Bubble Sort			$O(n^2)$	
Bubble Sort (Enhanced)			$O(n^2)$	
Merge Sort			$O(n \lg n)$	

Questions?